

EV Planner Spec v2

1. Goal and Motivation

EV stands for “Effort Value.” EVs increase a pokemon’s stats and allow a trainer to tune a pokemon for key battle situations, making its strengths shine. Every 4 EVs equate to 1 stat point at level 100, so reaching exact totals matters — especially for speed tie-breaks.

Objective: Minimize the number of moves needed to reach an exact EV total.

Moves are the unit of cost because they are easy to track in-game.

Worst-Case Example (No Optimizations)

- Each pokemon can earn at most 508 total EVs.
- Each pokemon defeated awards approximately 1–3 EVs.
- Training one pokemon: 170–508 pokemon defeated.
- Training a full team of six: 1,020–3,048 battles.

The sections below describe the mechanics and optimizations that reduce this cost.

2. Rules and Definitions

EV Caps

Scope	Cap
Per stat	252
Per pokemon (total)	508

Methods of Obtaining EVs

Battling: Each pokemon defeated yields EVs applied to one or more stats of the pokemon that defeated it.

Vitamins:

Property	Value
Cost	\$9,800
EV yield	10
Stat cap	Only effective while the target stat’s EVs are below 100

Items That Modify EV Yield

Item	Effect
Macho Brace	Doubles the base EVs earned by the holder in battle
Exp. Share	Grants the holder (out of battle) the same base EVs the battler received, before any item adjustments
Amulet Coin	Doubles money rewarded from trainer battles

Battle Encounter Nuances

- **VS Seeker (trainers):** Rematches available after a minimum of 100 steps (then multiples of 100). A trainer may decline a rematch.
- **Wild pokemon:** Encountered in fewer steps than trainer rematches, but encounter rate is normally less than 100%. A repel trick with a fainted lead pokemon can filter encounters to a specific species at effectively 100%.

Variable Glossary

All formulas below use these names consistently.

Variable	Type	Description
<code>opponent_ev_yield</code>	int	Base EVs awarded by defeating one opponent
<code>has_macho_brace</code>	bool	Whether the battler holds a Macho Brace
<code>macho_multiplier</code>	int	2 if <code>has_macho_brace</code> , else 1
<code>is_trained</code>	bool	Whether the battler is a high-level sweeper not receiving EVs
<code>has_exp_share</code>	bool	Whether a party member holds the Exp. Share
<code>has_amulet_coin</code>	bool	Whether the battler holds an Amulet Coin
<code>money_multiplier</code>	int	2 if <code>has_amulet_coin</code> , else 1
<code>money_rewarded</code>	int	Base money from a trainer battle
<code>vitamin_cost</code>	int	9,800
<code>vitamin_ev_yield</code>	int	10

Variable	Type	Description
<code>moves_per_ko</code>	int	Moves needed to knock out one opponent (assumed 1 under OHKO assumption)

3. EV Gain Model

EVs from Battle

The battler's item determines `battler_factor`:

```
if is_trained:
    battler_factor = 0          # sweeper: does not receive EVs itself
else:
    battler_factor = macho_multiplier # 1 (default) or 2 (Macho Brace)
```

```
shared_evs = opponent_ev_yield if has_exp_share else 0
```

```
ev_from_battle = (opponent_ev_yield * battler_factor) + shared_evs
```

Note: The Amulet Coin does not affect battle EVs — it only affects money. A pokemon in battle holds exactly one item, so the battler uses either Macho Brace, Amulet Coin, or neither (not both).

EVs from Money (Vitamins)

```
vitamin_cost = 9800
vitamin_ev_yield = 10
```

```
ev_from_money = ((money_rewarded * money_multiplier) // vitamin_cost) * vitamin_ev_yield
```

This represents EVs obtainable by converting earned money into vitamins.

Combined Metric: EV per Move

```
ev_per_move = (ev_from_battle + ev_from_money) / moves_per_ko
```

This ratio guides whether one battle option is better than another.

4. Battle Scenarios

Wild Pokemon Battles

```
moves_per_ko = 1 # OHKO assumption
```

```
wild_ev_per_move = ev_from_battle / moves_per_ko
```

Wild battles yield no money, so `ev_from_money` is always 0 for wild encounters.

Trainer Battles

A trainer battle involves defeating every pokemon in the opponent's party. The total EVs and money come from the full fight.

```
opponent_party = [pokemon_a, pokemon_b] # e.g. Nidorino, Sandshrew
```

```
def trainer_ev_per_move(opponent_party, player_party):
    total_ev = 0
    total_moves = 0
    for opponent in opponent_party:
        total_ev += ev_from_battle(opponent.ev_yield, player_party)
        total_moves += moves_per_ko
    total_ev += ev_from_money(money_rewarded, money_multiplier)
    return total_ev / total_moves
```

Trainer battles can be more efficient than wild battles because they also generate money convertible to vitamins.

5. Constraints

When Battle EVs Don't Count

Only count EVs from channels that still produce relevant EVs:

1. EVs to the battler
2. EVs to party members holding Exp. Share
3. EVs from money converted into vitamins (trainer battles only)

If the battler is trained (or already capped in the yielded stat), ignore battler EVs for that opponent. The battle is still useful when Exp. Share EVs or money EVs are useful.

```
useful_battler_evs = (not is_trained) and (battler_remaining_evs > 0)
useful_exp_share_evs = any(member.has_exp_share and member.remaining_evs > 0 for member in player_party)
useful_money_evs = (ev_from_money > 0) and any(0 <= member.current_stat_evs <= 90 for member in player_party)
```

```
battle_is_useful = useful_battler_evs or useful_exp_share_evs or useful_money_evs
```

EVs do not count when all three useful channels are false (for example, battler trained/capped, no useful Exp. Share recipient, and no useful vitamin target).

Vitamin Applicability

Vitamins only work while a stat's EVs are below 100.

```
can_apply_vitamin = (current_stat_evs < 100)
```

When to buy: `wallet >= vitamin_cost` and at least one party member has a stat with `0 <= current_stat_evs <= 90`.

Vitamin Batching

When planning ahead, account for future earnings. If future funds will be a known multiple of `vitamin_cost`, a party member can absorb vitamins as long as:

```
vitamin_ev_limit = 100
max_applicable = vitamin_ev_limit - current_stat_evs
max_vitamins = max_applicable // vitamin_ev_yield
```

6. Open Questions / TODOs

- ☐ Prove that a OHKO sweeper gains at least as many EVs-per-move as a 2HKO Macho Brace holder. Measure in number of operations as a heuristic; constants (encounter rate, battle open/close) can be ignored — focus on variables (moves, healing, status, recovery).
- ☐ Define a heuristic for whether a pokemon can OHKO:
 - Strong STAB move, super effective, battler level $\geq$ opponent level - 5
 - Strong STAB move, neutral hit, battler level $\geq$ opponent level + 10
 - If neither condition holds, use a trained sweeper instead.
- ☐ Design a Pokemon class that calculates EV yield.
- ☐ Design a PlayerParty class (item holding, current EV totals per member).
- ☐ Build the worst-case example end-to-end through all optimizations (compare no-optimization baseline to each incremental improvement).

7. Future Considerations

- **Notebook conversion:** Rewrite formulas and scenarios as a Jupyter notebook — markdown cells for explanation, code cells for computation, reusable functions importable from a shared module.
- **Time-based metric:** Supplement EV-per-move with a real-time estimate (encounter rate, animation time, step count).
- **Full party optimization:** Extend from single-member training to planning an optimal route for all six party members simultaneously.
- **Data sourcing:** Gather encounter tables, trainer rosters, money rewards, and EV yields from authoritative game data.